

project and resume based -

Microservices (what each one does)

API Gateway

Single entry point. Routes requests to downstream services, applies rate limiting, request/response mapping, and passes identity (JWT) to services.

Auth Service

OAuth2/JWT issuance and validation, user/role management, token introspection. Provides policies/permissions consumed by the gateway and services

Employee Management Service

Source of truth for employee profiles, grades/bands, cost centers, employment status, banking details, tax identifiers. Publishes employee.events on changes.

Timekeeping Service

Captures hours, shifts, overtime, leave, and holiday calendars. Exposes REST to query time data; publishes time.events (e.g., approved timesheets).

Crew Planning Service

Manages crew rosters, allocations and rules specific to railway operations (night shifts, mandatory rest, union constraints). Supplies schedules to payroll/payslip.

Payslip Service

Domain logic for earnings & deductions: base pay, allowances, holiday pay, overtime, arrears, retro adjustments. Consumes time/employee/crew data and produces itemized payslip breakdowns.

Payroll Service (orchestrator)

Coordinates end-to-end payroll runs (per period, per business unit). Calls Payslip, aggregates totals, applies compliance checks, triggers disbursement, and emits payroll.events (success/failure, per-employee results).

Notification Service

Listens for payroll outcomes and notification commands; sends emails/SMS/in-app notifications with payslip PDFs/links. Manages templates and delivery status.

Reporting Service

BI/analytics and statutory reports (ledger exports, cost center summaries, audit trails). Subscribes to payroll.events for near-real-time dashboards; offers REST for ad-hoc queries via the gateway.

In this migration program, we had around 6–7 feature teams, each owning specific services. I was part of the Payslip module team with about 4 members, including one lead and one senior dev and two j-dev. Our responsibility was to migrate 26 PL/SQL procedures into Java/Spring Boot microservices, integrate with Employee, Timekeeping, and Crew Planning services, and ensure accuracy in payroll calculations.

-- security

To secure our microservices, we had a centralized Auth service issuing JWTs after user authentication. All requests passed through an API Gateway, which validated tokens and enforced authorization. The gateway attached the JWT to each internal call so downstream services could trust the identity and claims without re-authenticating. For service-to-service calls, we used token-based trust and secured all traffic over HTTPS. Kafka topics were ACL-protected. This setup ensured a single source of truth for identity while keeping the services decoupled and secure.

We implemented centralized authentication using Spring Security OAuth2. Our Auth service issued signed JWTs. The API Gateway, built on Spring Cloud Gateway, had a filter to validate tokens and reject unauthorized calls. Each microservice was a Spring Boot app configured as a resource server, validating JWTs using the Auth service's public key. Role-based access was enforced via @PreAuthorize annotations. For internal calls, we either propagated the user token or used a service account token. This allowed consistent, secure, and decoupled access control across the system.

For non-production, everything was on-prem due to client policies—we had Jenkins, SonarQube, and containerized deployments on internal servers. For production, we moved

to EKS. Jenkins handled the pipeline end-to-end: build, containerize, push to ECR, and deploy to Kubernetes using Helm. Each microservice ran as its own deployment, 4–6 pods in prod, autoscaling as needed. This hybrid approach gave us compliance in dev/test and scalability in prod. Our Jenkins automation reduced manual effort by ~25%, which I've highlighted in my resume.

In our migration project, the payslip generation process starts whenever the Payroll service triggers a request for a particular pay cycle. The Payslip microservice is then responsible for fetching and processing all the necessary data to generate an accurate payslip.

First, we gather the employee's details using components like `GetEmployees` and `GetClaimProfile` – this includes personal and financial data, grade, pay band, and any claims or entitlements. We then validate time and attendance records via modules like `CheckPriorTimeslips` and `CheckBridging`. This step ensures we capture working days, overtime hours, holiday shifts, and also avoid duplicate or missing entries.

Once data is validated, we move to earnings calculation. Using modules like `ProcessEmployee`, `ProcessEntitlement`, and `AddNwslip`, we calculate basic salary based on pay band and working hours, apply overtime multipliers, add holiday pay, and include allowances such as travel or meal perks if applicable.

Next, we compute deductions. This covers income tax, statutory contributions, loans or advances, and leave without pay. These updates are handled through modules like `InsertEmplHist` and `UpdateEmplHistTpnw` to maintain accurate history records. If any discrepancies are found, they're logged using `InsertError` for review.

Finally, we consolidate earnings and deductions to get the net pay, update the payroll history, and generate the final payslip. At this stage, events are published to the Notification service to send the payslip PDF or email to the employee and to the Reporting service for audits and compliance.

This entire flow ensures every payslip reflects correct data – employee info, time, entitlements, and compliance – all migrated from PL/SQL procedures into modular, Spring Boot-based services. My specific responsibility was to design and implement the holiday and payslip modules, which included around 26 procedures.

kafka

1) Two modes of payroll invocation

a. Ad-hoc / single-employee request

Payroll service calls Payslip service synchronously via REST:

POST /payslip/generate with body { employeeId, payPeriod, requestId }.

Payslip handles end-to-end calculation for that employee and returns immediate result (success/failure).

For audit/async downstream work (reporting, notification), Payslip publishes a payslip.generated event to Kafka after finalization.

b. Batch / mass run (all employees)

Payroll service starts a batch run and publishes a payroll.run.requested event to Kafka containing run metadata (payPeriod, runId, filter criteria).

Multiple consumers (Payslip instances) subscribe to this topic and process per-employee messages or the Payroll service fans out per-employee payslip.generate.command events for parallel processing.

Each Payslip instance processes its assigned employee, emits payslip.generated or payslip.failed events, and the Payroll orchestrator (or a consumer) aggregates completion status.

Use case: ad-hoc for corrections / employee queries; batch for scheduled payroll cycles.

2) Kafka topics & sample message bodies

(Use compact, versioned topic names; payloads JSON/Avro; schema in Schema Registry)

Topics

employee.events — employee.create/update/delete

time.events — approvedTimesheet / leaveApproved / shiftWorked

payroll.run.requests — batch payroll run started (contains runId)

payslip.commands — direct commands to generate a payslip for a specific employee

payslip.results — success / failure with details (used by Reporting & Notification)

notification.commands — requests to send email/SMS with payload or attachment reference

Example: payroll.run.requests

```
{  
  runId: run-2025-08-01-01,  
  payPeriod: { from:2025-07-01,to:2025-07-31},  
  initiatedBy:payroll-user,  
  scope:ALL_ACTIVE,  
  timestamp:2025-08-01T02:00:00Z,  
  correlationId:corr-1234  
}
```

Example: payslip.commands (fan-out from Payroll)

```
{  
  commandId: cmd-emp-9876,  
  runId: run-2025-08-01-01,  
  employeeId: EMP9876,  
  payPeriod: {from:2025-07-01,to:2025-07-31},  
  requestedBy:payroll-service,  
  timestamp:2025-08-01T02:01:12Z
```

```
}
```

Example: payslip.results (success)

```
{  
  resultId:res-emp-9876,  
  commandId:cmd-emp-9876,  
  runId:run-2025-08-01-01,  
  employeeId:EMP9876,  
  status:SUCCESS,  
  netPay:4100.00,  
  payslipRef:s3://payslips/2025-07/EMP9876.pdf,  
  calculationSummary: {  
    basic:4000.0, overtime:300.0, holidayPay:300.0, allowances:200.0,  
    deductions:700.0  
  },  
  timestamp:2025-08-01T02:01:59Z  
}
```

Example: payslip.results (failure)

```
{  
  resultId:res-emp-1234,  
  commandId:cmd-emp-1234,  
  runId:run-2025-08-01-01,  
  employeeId:EMP1234,  
  status:FAILED,  
  errorCode:TIME_DATA_MISSING,
```

```
errorMessage:No approved timesheet found for payPeriod,  
attempts:3,  
timestamp:2025-08-01T02:03:10Z  
}
```

Example: notification.commands

```
{  
  notificationId:notif-4567,  
  to:employee@example.com,  
  channel:EMAIL,  
  template:payslip_ready,  
  templateParams:{employeeId:EMP9876,payslipUrl:https://...},  
  priority:NORMAL,  
  correlationId:res-emp-9876  
}
```

3) Microservice pattern: Saga choreography (primary) + local idempotent processing

Why choreography:

We favored event-driven choreography for payroll because it scales horizontally for large batches, reduces a single orchestrator bottleneck, and fits well with existing Kafka infrastructure. Payroll fans out payslip.commands and downstream services react to events independently.

How it worked in practice:

Payroll publishes payslip.commands.

Payslip service consumes commands and performs a local transaction: read employee/time data (from own DB or via synchronous calls), compute payslip, persist payslip, publish payslip.results.

Reporting and Notification consume payslip.results.

When a compensation is needed (e.g., reverse an already settled payslip), a compensating event payslip.reversal.requested is published and consumers apply compensating actions (mark reversed, adjust ledger, notify employee).

When orchestration is used:

For cross-cutting transactional needs where a single logical flow must be monitored (e.g., disbursement with bank integration that needs final settlement), a lightweight Payroll Orchestrator service monitors payslip.results and triggers downstream settlement steps — effectively an orchestrator for specific sub-flows, not to replace choreography.

4) Internal communication with Timekeeping & Employee Management

Principles

Source of truth: Employee Management owns employee data. Timekeeping owns time data. Payslip reads via REST when it needs the latest live data, but also consumes event streams to maintain a local cache for performance and resiliency.

Hybrid communication:

Async-first: Payslip subscribes to employee.events and time.events to keep a local, read-optimized store (CQRS pattern). This allows near-real-time updates without many synchronous calls during batch runs.

REST fallback: For one-off ad-hoc checks or the latest authoritative data (e.g., pending HR change not yet in event stream), Payslip calls Employee Management or Timekeeping REST endpoints.

Example flows

During batch run: Payslip consumer reads `payslip.commands` → uses `employeeId` to query local cache (or REST fallback) → uses cached time entries (or REST fallback) → compute payslip → write to Payslip DB → publish `payslip.results`.

Timekeeping late-arrival: If Timekeeping publishes a `time.events` late (after payslip already generated), a reconciler job detects divergence (e.g., via sequence numbers or versioning) and publishes `payslip.recalc.command` for affected employees.

5) Reliability, ordering, idempotency, retries, and error handling

Idempotency: Each `payslip.command` contains a `commandId`. Payslip service persists processed `commandIds` to ensure repeated deliveries (Kafka redeliveries) do not create duplicates.

Exactly-once semantics (application-level): Use idempotent writes and compare-and-set for critical updates. If Kafka + DB transactions supported, use Kafka transactions (where available) to atomically commit produced events with consumed offsets.

Retries: Consumer configs with exponential backoff for transient errors. For non-transient errors (missing data), publish `payslip.failed` with error code and let a human/automated retry process reissue a corrected `payslip.command`.

Dead-letter queue (DLQ): Messages failing after N attempts go to `payslip.dlq` for manual intervention.

Compensation: For irreversible side effects (bank disbursements), we modeled compensating events (e.g., `disbursement.reversal.requested`) so downstream systems can reconcile.

6) How Payslip publishes generation status to Notification

After finalizing a payslip, Payslip publishes `payslip.results` (SUCCESS/FAILED).

Notification service consumes `payslip.results` and maps it to `notification.commands` (templating, channeling, rate limiting).

For success: Notification sends payslip link or attached PDF; for failures: sends alert to payroll admins.

Additionally, Payslip may directly call Notification REST for high-priority alerts (sync), but we preferred event-driven approach for bulk.

7) Spring Boot / Spring Kafka implementation details (concise)

Producer: `KafkaTemplate<String, PayslipCommand` with `acks=all`, retries configured.

Consumer: `@KafkaListener(topics = payslip.commands)` with manual ack or concurrency set to match pods.

Schema: Use Avro + Schema Registry for forward/backward compatibility or JSON + versioning.

Transactional processing: Use Spring Kafka transactions to ensure event publishing and DB update atomicity where possible.

Concurrency: Set concurrency on `ConcurrentMessageListenerContainer` to match pods \ partitions for parallelism.

8) Partitioning & pods sizing guidance (performance)

Partitioning: Partition `payslip.commands` by `runId` or `employeeId` hash to spread load and preserve ordering per employee.

Pods: For peak payroll runs (10–13K users, large batches), scale Payslip service pods so that consumers \ partitions provides enough parallelism. Example: 12 partitions + 6 pods with concurrency 2.

Backpressure: Monitor consumer lag and scale HPA on lag metrics.

9) Ready-to-speak summary (2–3 sentences)

We supported two invocation modes: synchronous REST for ad-hoc single-employee payslips and event-driven batch runs via Kafka for full payroll cycles. Payroll publishes per-employee `payslip.commands` and Payslip consumes, calculates (using cached employee/time data or REST fallback), persists results, and emits `payslip.results`. We used a choreography-style saga with idempotent consumers, DLQs, retry/backoff, and compensating events for reversals. Notification and Reporting consume the results to deliver payslips and dashboards.

fail

1. Idempotent and Stateful Processing

Every request/message has a unique ID (e.g., commandId or combination of runId + employeeId).

The Payslip service stores processing metadata in its database (or a dedicated table) with states like PENDING, PROCESSING, SUCCESS, FAILED.

When a message is reprocessed, the service checks this table; if the state is already SUCCESS, it skips; if FAILED, it retries.

This ensures at-least-once delivery doesn't create duplicates.

Example schema:

payslip_run_status

(runId, employeeId, commandId, status, retryCount, lastUpdated, errorCode)

2. Batch Run Continuity

Batch runs are resumable:

If processing stops at employee 1342, the next batch iteration starts by querying all PENDING or FAILED records for that runId.

Only unprocessed employees are picked, so you don't repeat work.

Kafka replay: Topics are partitioned and offsets are managed; if the consumer fails mid-run, Kafka redelivery or a manual replay reprocesses only messages not acknowledged.

3. Error Handling Strategy

Transient failures (DB connection, temporary service down): Retry with exponential backoff (e.g., 3–5 attempts).

Business errors (missing time data, invalid employee): Mark as FAILED with error code; produce an event to a DLQ or payslip.failed topic. These can be corrected and re-triggered later.

Poison messages: Sent to a Dead Letter Queue after max retries for manual intervention.

4. Retry & Compensation

Manual or automated retry: Once errors are corrected (e.g., missing timesheet fixed), rerun only the FAILED employees by publishing new commands or by a scheduled retry job.

Compensating actions: If a payslip was generated with wrong data, publish a payslip.reversal.requested event and regenerate.

5. Ensuring Delivery & Tracking

Audit tables and dashboards: Payroll admins can see processed/failed/pending counts per run.

Offset commits: Kafka commits offsets only after successful DB update and event publishing (using Spring Kafka transactions or outbox pattern).

Outbox pattern: In some designs, DB changes and outgoing events are written together to an outbox table, then published reliably.

Ready-to-speak answer (short):

To handle 10–15K employees per run reliably, we made the process idempotent. Each message has a unique runId + employeeId key, and we persist status (PENDING, SUCCESS, FAILED). If a failure occurs at employee 1342, the run can be resumed; the service picks only unprocessed employees. Kafka retries transient errors automatically, and permanent failures go to a DLQ. Admins can fix data and re-trigger just the failed employees, ensuring no duplicates or missed payslips.

The client wanted to migrate from PL/SQL to Java Spring Boot mainly to modernize the system. The old solution had hundreds of stored procedures tightly coupled to Oracle, which made changes slow and required niche skills. By moving to Spring Boot microservices, we gained better scalability—logic runs outside the DB so we can scale pods instead of overloading the database. It's also easier to integrate with other systems using REST APIs and Kafka, automate testing and CI/CD, and prepare for cloud deployment. It reduced vendor lock-in and made the system more maintainable and future-ready.

1. Are you processing one employee at a time or multiple? How?

“We process employees in parallel to handle 10–15K records efficiently. Instead of doing one-by-one sequential processing, we split the batch into smaller chunks and process multiple employees at once:

The Payroll service breaks a run into partitions and publishes Kafka events per employee or per group.

The Payslip microservice has multiple consumers and worker threads (Spring Boot with `@Async` /thread pools`) to process messages in parallel.

The database updates are handled transactionally per employee, so even if one fails, others continue.

This approach gives us throughput without blocking, and each transaction is isolated to that employee's record.”

2. How do you ensure database consistency?

“We follow strong consistency at the employee level. Each payslip calculation and DB update is wrapped in a single transaction, so either it completes fully or rolls back.

For messaging, we use Kafka with offset commits after DB success to avoid duplicates.

We designed the processing to be idempotent using unique keys (`runId + empld`), so retries don’t cause double processing.

For large updates, we use batch inserts/updates when possible, but still keep each employee isolated for correctness.”

3. PL/SQL was faster inside the DB. How did you achieve similar performance in Java?

“Yes, stored procedures run inside the DB engine, so they’re very fast. But they don’t scale well. To match performance, we used multiple techniques:

Parallelism: Process many employees at the same time via Kafka partitions and worker threads.

Optimized queries: We reused tuned SQL and indexes, reduced unnecessary joins, and fetched data in bulk where possible.

Caching: Frequently used data like pay bands or holiday rules were cached in memory.

Horizontal scaling: Java services run on multiple pods in EKS, so throughput scales as we add instances.

This way, even though we moved logic out of the DB, overall processing time is competitive and can handle growth better.”

1. Monitoring and Observability

“We implemented proper observability because payroll is critical. Each microservice exposes health and metrics endpoints, which we monitor using Prometheus and visualize on Grafana dashboards. Application and batch logs are aggregated using ELK/Splunk for searching errors. We also have alerts configured – for example, if a Kafka consumer lags or

a payroll run takes longer than expected, the team gets notified. This helped us catch issues before they impacted users.”

2. Data Validation and Quality

“Since this was a migration, ensuring correctness was key. We ran the Java microservices in parallel with the legacy PL/SQL system during testing and compared outputs. We used reconciliation scripts to validate payslips, taxes, deductions, and totals. Also, we wrote unit tests and integration tests using JUnit and Mockito, and created automated regression suites to ensure every change didn’t break payroll logic.”

3. Compliance and Audit

“Payroll deals with sensitive PII, so we followed strict security. Sensitive data is encrypted at rest and in transit (TLS). Access is role-based, controlled via a centralized Auth service and API Gateway. We implemented audit logging to track who triggered a payroll run, changes to employee records, and any failed transactions. These logs are immutable and can be reported for compliance or audits.”

4. Edge Cases

“We also had to handle exceptions like mid-cycle joiners/leavers, retroactive adjustments, off-cycle payroll runs, and reversals. The system is flexible – we can rerun payslips for a specific employee or batch, and we designed APIs/events to handle corrections without impacting the entire run. This was important for accuracy and user trust.”

5. Non-Functional Requirements

“We had clear SLAs – a payroll run for 10–15K employees needed to finish within the scheduled window. To meet this, we optimized queries, used parallel processing (Kafka consumers, thread pools), and scaled pods on EKS as needed. We load tested the services using JMeter and tuned the DB indexes and batch sizes. The result was consistent performance under load.”

6. Team Delivery Practices

“We worked in an Agile setup. Each module – payslip, employee management, timekeeping, etc. – was handled by a small squad of 4–6 people (dev, QA, lead). We had sprint planning, daily standups, backlog grooming, and demos to stakeholders. Since modules interacted (e.g., payslip needed timekeeping data), we used integration tests and regular syncs across teams to handle dependencies and avoid blockers.”